

Renewable energy-Wind

Model Tuning

02/01/2026

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model performance summary for hyperparameter tuning.
- Model building with pipeline
- Appendix

Executive Summary

- Actionable insights & recommendations

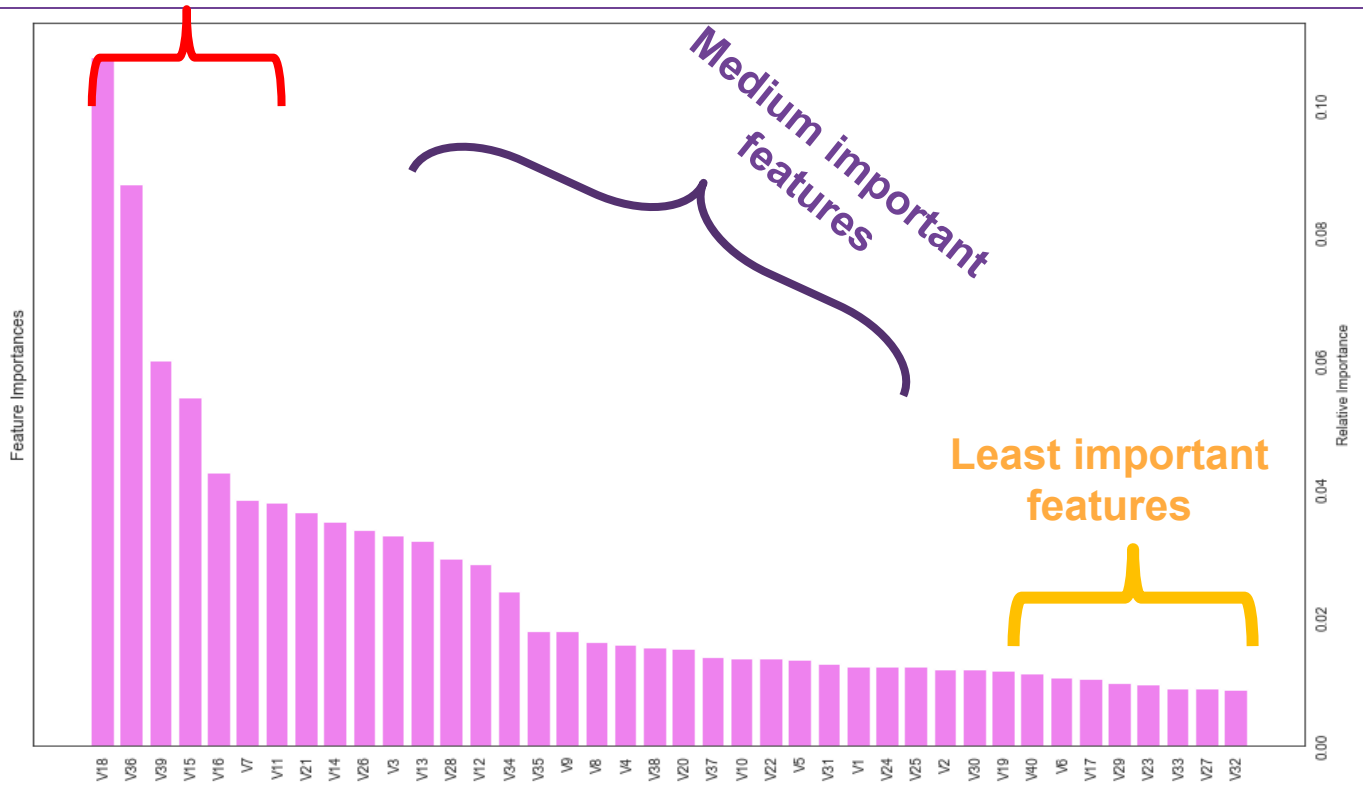
The most important features are as listed V18,V36,V39,V15,V16

The least import features: V32,V22,V27,V29

- We recommend to keep eyes on the most important feature and tracking the variation in their value over time. Features with distribution skewed to the left toward negative values are more likely to be an indicator for failure of wind turbines
- Keep the most important feature sensors in great condition. Frequently inspect these sensors because any inefficiency in these sensors could cost a lot while make less frequent inspection for medium important features.
- Creating automatic alarm, which will start when values of the most important features becomes negative for following many days.
- Future work: with gathering more data, we recommend rebuild the models and tuning the hyperparameters using cloud or GPU for improved results and higher Recall score.

Executive Summary

**Most important
features**

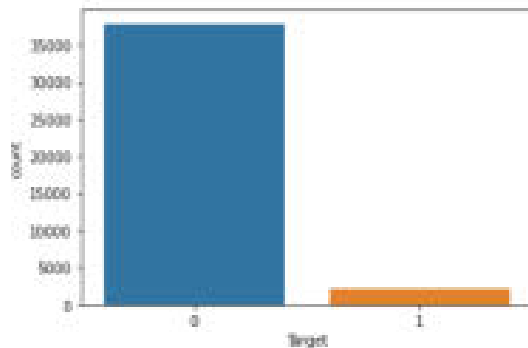


Business Problem Overview and Solution Approach

- Problem: The cost of repairing a generator is much less than the cost of replacing it, and the cost of inspection is less than the cost of repair. If component failure can be predicted accurately and the component is replaced before it fails, the costs of operation and maintenance will be much lower.
- Object: Identify component failures so that the generators could be repaired before failing/breaking to reduce the overall maintenance cost.
- The solution approach / methodology: Preparing the data; Build various classification models; dealing with imbalanced data set; build model with undersampled dataset and oversampled dataset; Tuning the models; Find the best model that will have generalized result; Pipelines to put the final model to production.

EDA Results

- "Target" class is imbalanced with 37813 or 94.53% "No failures (i.e., 0)" and 2187 or 5.47% "Failures (i.e., 1)"

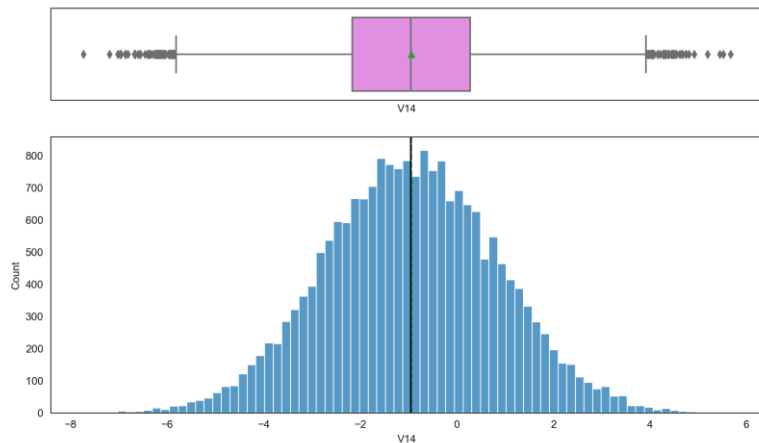


- Median of the attribute was used to impute any missing values in the attributes (i.e., for "V1" and "V2" which had 46 & 39 missing values)
- This was performed separately on training set and validation set to prevent any data leak!

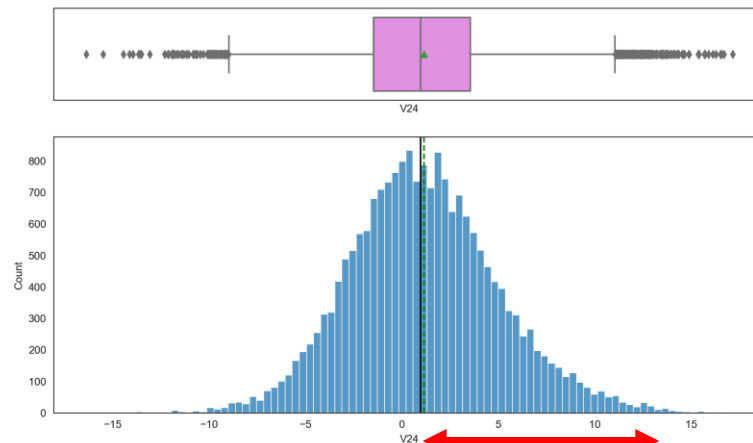
[Link to Appendix slide on data background check](#)

EDA Results

- Please mention the key results from EDA
- The variance of the independent features varies across the features .
 - E.g. V18,V14 have small variance while V24,V16,V8 have wide variance



4



15

[Link to Appendix slide on data background check](#)

Model Evaluation Criteria & Model Building

- Creation of a customized metric:

(Minimum Cost)/(Cost Associated with the Machine Learning model)

Higher the value of the customized metric, better the performance of the model

- 7 different machine learning algorithms were fit on original training dataset

- Logistic Regression
- Decision Tree
- Random Forest
- Bagging Classifier
- Boosting Classifiers (AdaBoost, Gradient Boost, XGBoost)

*** 5 fold cross validation was performed to calculate an average customized metric score on training data set**

- Class imbalance was handled by

- Synthetic Minority Oversampling Technique (SMOTE)
- Random Under sampler

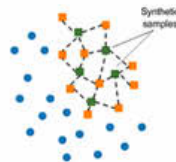
- & the 7 machine learning algorithms were fit again
- **5 fold cross validation was performed to calculate an average customized metric score on training dataset**

- Model performance was evaluated on validation data set to assess if model generalizes well or is prone to overfitting/underfitting

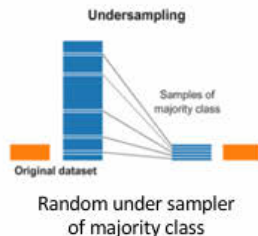
		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Customized Metric:

$$((TP + FN) * 15) / (TP * 15 + FP * 5 + FN * 40)$$



SMOTE – adding synthetic minority class data points



Data Preprocessing

- Duplicate value check
 - No duplication
- Missing value treatment
 - There are missing values in the first two columns
 - We applied SimpleImputer with median on data set
- Outlier check (treatment if needed)
 - No outlier,
- Feature engineering
 - All the feature are numerical values of sensors. No need to feature engineering.
- Data preparation for modeling
 - Split data to three parts: train(15000), validation(5000), and test.(5000)

Model Performance Summary

Which metric to optimize?

- * We need to choose the metric which will ensure that the maximum number of generator failures are predicted correctly by the model.
- * We would want Recall to be maximized as greater the **Recall**, the higher the chances of minimizing false negatives.
- * We want to minimize false negatives because if a model predicts that a machine will have no failure when there will be a failure, it will increase the maintenance cost.

Model Performance Summary

Training	Accuracy	Recall	Precision	F1
Gradient Boosting tuned with oversampled data	0.990	0.989	0.992	0.990
AdaBoost classifier tuned with oversampled data	0.989	0.982	0.995	0.989
Random forest tuned with undersampled data	0.967	0.935	0.999	0.966
XGBoost tuned with oversampled data	0.998	1.000	0.996	0.998

Validation	Accuracy	Recall	Precision	F1
Gradient Boosting tuned with oversampled data	0.963	0.880	0.594	0.709
AdaBoost classifier tuned with oversampled data	0.983	0.873	0.819	0.845
Random forest tuned with undersampled data	0.942	0.899	0.488	0.633
XGBoost tuned with oversampled data	0.974	0.899	0.709	0.793

Winner

Random Forest model with undersampled data has an acceptable Recall score comparing with other models. All the other models show overfit.

Productionize and test the final model using pipelines

- steps taken to create a pipeline for the final model
 - Pipeline contains:
 - 1. Imputing the missing values using median strategy,
 - 2. The estimator. To fit and predict the model (Tuned Random Forest model)
- Summary of the performance of the model built with pipeline on test dataset

Test	Accuracy	Recall	Precision	F1
Pipeline	0.988	0.858	0.931	0.893

- Summary of most important factors used by the model built with pipeline for prediction
 - SimpleImputer (**strategy="median"**) was applied first.
 - For RadnomForest model: max_features='sqrt',random_state=1,max_samples=0.6,n_estimators=200, min_samples_leaf=2)
 - We imputed the missing values of the dataset before applying the resampling method that used to training the pipelines model.
[Link to Appendix slide on model assumptions](#)

APPENDIX



Happy Learning !



